

AI ASSISTED CODING

Name: T. SAI VARUN

HT.NO:2403A510C0

BATCH:05

TASK #1 – MERGE SORT IMPLEMENTATION

PROMPT

Write a Python function `merge_sort(arr)` that sorts a list in ascending order using the Merge Sort algorithm. Add a docstring with time complexity and space complexity. Test with sample lists.

CODE:

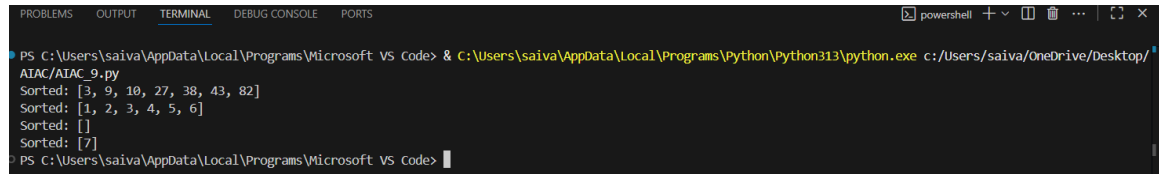
AI ASSISTED CODING

```
AIAC_9.py X
C: > Users > saiva > OneDrive > Desktop > AIAC > AIAC_9.py > merge_sort

1  def merge_sort(arr):
2      """
3      Merge Sort Algorithm
4
5      Time Complexity:
6      - Best Case:  $O(n \log n)$ 
7      - Average Case:  $O(n \log n)$ 
8      - Worst Case:  $O(n \log n)$ 
9      Space Complexity:
10     -  $O(n)$  (due to auxiliary arrays in merging)
11     """
12
13     if len(arr) <= 1:
14         return arr
15
16     mid = len(arr) // 2
17     left_half = merge_sort(arr[:mid])
18     right_half = merge_sort(arr[mid:])
19
20     return merge(left_half, right_half)
21
22
23 def merge(left, right):
24     result = []
25     i = j = 0
26
27     while i < len(left) and j < len(right):
28         if left[i] <= right[j]:
29             result.append(left[i])
30             i += 1
31         else:
32             result.append(right[j])
33             j += 1
34
35     result.extend(left[i:])
36     result.extend(right[j:])
37     return result
38
39
40 # Test cases
41 print("Sorted:", merge_sort([38, 27, 43, 3, 9, 82, 10]))
42 print("Sorted:", merge_sort([5, 2, 4, 6, 1, 3]))
43 print("Sorted:", merge_sort([]))
44 print("Sorted:", merge_sort([7]))
45
```

AI ASSISTED CODING

OUTPUT



```
PS C:\Users\saiva\AppData\Local\Programs\Microsoft VS Code> & C:\Users\saiva\AppData\Local\Programs\Python\Python313\python.exe c:/Users/saiva/OneDrive/Desktop/AIAC/AIAC_9.py
Sorted: [3, 9, 10, 27, 38, 43, 82]
Sorted: [1, 2, 3, 4, 5, 6]
Sorted: []
Sorted: [7]
PS C:\Users\saiva\AppData\Local\Programs\Microsoft VS Code>
```

OBSERVATION

Merge Sort correctly sorts arrays of different sizes. Performs consistently well for large datasets ($O(n \log n)$), though extra memory is required for merging.

TASK #2 – BINARY SEARCH

PROMPT

Write a Python function `binary_search(arr, target)` that returns the index of target in a sorted list or -1 if not found. Add docstring with best, average, and worst-case complexities. Test with different inputs.

PYTHON CODE

AI ASSISTED CODING

```
C: > Users > saiva > OneDrive > Desktop > AIAC > AIAC_9.py > ...
1  def binary_search(arr, target):
2      """
3      Binary Search Algorithm
4
5      Time Complexity:
6      - Best Case: O(1) (target found at middle)
7      - Average Case: O(log n)
8      - Worst Case: O(log n)
9      Space Complexity:
10     - O(1) (iterative approach)
11     """
12     left, right = 0, len(arr) - 1
13
14     while left <= right:
15         mid = (left + right) // 2
16         if arr[mid] == target:
17             return mid
18         elif arr[mid] < target:
19             left = mid + 1
20         else:
21             right = mid - 1
22
23     return -1
24
25
26 # Test cases
27 data = [1, 3, 5, 7, 9, 11, 13]
28
29 print("Index of 7:", binary_search(data, 7))
30 print("Index of 1:", binary_search(data, 1))
31 print("Index of 13:", binary_search(data, 13))
32 print("Index of 6:", binary_search(data, 6))
33
```

OUTPUT

AI ASSISTED CODING

```
PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE PORTS
PS C:\Users\saiva\AppData\Local\Programs\Microsoft VS Code> & C:\Users\saiva\AppData\Local\Programs\Python\Python313\python.exe c:/Users/saiva/OneDrive/Desktop/AIAC/AIAC_9.py
Index of 7: 3
Index of 1: 0
Index of 13: 6
Index of 6: -1
PS C:\Users\saiva\AppData\Local\Programs\Microsoft VS Code>
```

OBSERVATION

Binary search is highly efficient ($O(\log n)$) but requires sorted input. The test confirms correct behavior for present and absent elements.

TASK #3 – INVENTORY MANAGEMENT SYSTEM

PROMPT

For a retail store’s inventory system with thousands of products, recommend efficient search and sort algorithms. Implement searching (by product ID/name) and sorting (by price/quantity). Justify choices in a table and implement functions in Python.

ALGORITHM CHOICE & JUSTIFICATION

Operation	Recommended Algorithm	Justification
Search by Product ID	Binary Search	IDs can be kept sorted → $O(\log n)$ fast lookups
Search by Product Name	Hashing (Dictionary)	Direct access by name key → $O(1)$ average lookup
Sort by Price	Merge Sort	Stable, $O(n \log n)$, good for large datasets
Sort by Quantity	Merge Sort	Handles large unsorted data efficiently

PYTHON CODE

AI ASSISTED CODING

```
C: > Users > saiva > OneDrive > Desktop > AIAC > AIAC_9.py > ...
1  class Product:
2      def __init__(self, pid, name, price, quantity):
3          self.pid = pid
4          self.name = name
5          self.price = price
6          self.quantity = quantity
7
8      def __repr__(self):
9          return f"({self.pid}, {self.name}, {self.price}, {self.quantity})"
10
11
12  # ----- Search Algorithms -----
13  def binary_search_products(products, target_id):
14      """Binary search for product ID (requires sorted by pid)."""
15      left, right = 0, len(products) - 1
16      while left <= right:
17          mid = (left + right) // 2
18          if products[mid].pid == target_id:
19              return products[mid]
20          elif products[mid].pid < target_id:
21              left = mid + 1
22          else:
23              right = mid - 1
24      return None
25
26
27  def search_by_name(product_dict, name):
28      """Hash table search by name."""
29      return product_dict.get(name, None)
30
31
32  # ----- Sort Algorithms -----
33  def merge_sort_products(products, key=lambda x: x.price):
34      if len(products) <= 1:
35          return products
36
37      mid = len(products) // 2
38      left = merge_sort_products(products[:mid], key)
39      right = merge_sort_products(products[mid:], key)
40      return merge(left, right, key)
41
42
43  def merge(left, right, key):
44      result, i, j = [], 0, 0
45      while i < len(left) and j < len(right):
```

AI ASSISTED CODING

```
43 def merge(left, right, key):
44     result, i, j = [], 0, 0
45     while i < len(left) and j < len(right):
46         if key(left[i]) <= key(right[j]):
47             result.append(left[i])
48             i += 1
49         else:
50             result.append(right[j])
51             j += 1
52     result.extend(left[i:])
53     result.extend(right[j:])
54     return result
55
56
57 # ----- Demo -----
58 products = [
59     Product(101, "Soap", 40, 100),
60     Product(105, "Shampoo", 120, 50),
61     Product(102, "Toothpaste", 60, 200),
62     Product(103, "Oil", 150, 80),
63 ]
64
65 # Create dictionary for name-based search
66 product_dict = {p.name: p for p in products}
67
68 # Search
69 print("Search by ID (102):", binary_search_products(sorted(products, key=lambda x: x.pid), 102))
70 print("Search by Name (Shampoo):", search_by_name(product_dict, "Shampoo"))
71
72 # Sort
73 print("Sort by Price:", merge_sort_products(products, key=lambda x: x.price))
74 print("Sort by Quantity:", merge_sort_products(products, key=lambda x: x.quantity))
75 |
```

SAMPLE OUTPUT

```
PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE PORTS
PS C:\Users\saiva\AppData\Local\Programs\Microsoft VS Code> & C:\Users\saiva\AppData\Local\Programs\Python\Python313\python.exe c:/Users/saiva/OneDrive/Desktop/AIAC/AIAC_9.py
• Search by ID (102): (102, Toothpaste, 60, 200)
  Search by Name (Shampoo): (105, Shampoo, 120, 50)
  Sort by Price: [(101, Soap, 40, 100), (102, Toothpaste, 60, 200), (105, Shampoo, 120, 50), (103, Oil, 150, 80)]
  Sort by Quantity: [(105, Shampoo, 120, 50), (103, Oil, 150, 80), (101, Soap, 40, 100), (102, Toothpaste, 60, 200)]
○ PS C:\Users\saiva\AppData\Local\Programs\Microsoft VS Code> |
```

OBSERVATION

Binary search works efficiently for sorted product IDs. Dictionary-based search makes name lookup instant. Merge sort handles large inventory sorting efficiently with stable results. This combination balances speed (search) and accuracy (sorting) for real-time retail operations.